

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №X
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Е. А. Самсонова
Группа: М8О-304БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Познакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научиться работать с матрицами перспективы, ортографической проекцией и аффинными преобразованиями.

Вариант №6: пирамида

Дополнительные задания:

1. Обеспечить переключение матрицы проекции (ортографическая и перспективная) в интерфейсе.
2. Добавить элементы интерфейса для изменения позиции, поворота и растяжения фигуры.
3. Добавить возможность движения и поворота фигуры по сложной траектории, сделать элемент интерфейса для паузы и воспроизведения “анимации”, а также возможность изменять скорость “анимации” и другие параметры траектории (например радиус траектории и др.).
4. Добавить возможность изменять цвет фигуры с помощью элементов интерфейса (в ImGui есть ColorEdit для этого).
5. Назначить процедурно цвета для каждой вершины, например в зависимости от позиции вершин в локальной системе координат фигуры, тогда возможность изменять цвет с помощью элементов интерфейса будет умножать выбранный цвет на цвет вершин.
6. Подумать, как можно нарисовать больше одного объекта на сцене используя больше одного набора дескрипторов (VkDescriptorSet).

Метод решения

В работе используется API Vulkan совместно с библиотеками GLFW (создание окна и обработка ввода), ImGui (построение графического интерфейса) и GLM (линейная алгебра). Геометрия пирамиды задается в локальной системе координат массивом из пяти вершин: одна вершина в верхушке и четыре в основании, образующем квадрат. Для отрисовки граней используется индексный буфер, описывающий шесть треугольников – 4 боковые грани и 2 треугольника основания. Данные вершин и индексов загружаются в GPU через буферы, создающиеся библиотекой VMA с флагом, который позволяет сразу записывать данные с CPU.

Для передачи данных в шейдеры используется uniform-буфер, содержащий три матрицы (model, view, proj) и вектор цвета. Матрица model

выполняет аффинные преобразования объекта (перенос, поворот, масштаб), матрица `view` задается через `glm::lookAt` и переводит вершины в систему координат камеры, а матрица `proj` строится либо через `glm::perspective` для перспективной проекции, либо через `glm::ortho` для ортографической. Итоговое положение вершины вычисляется в вершинном шейдере по формуле `proj * view * model * vec4(position, 1.0)`. При работе с Vulkan учитываются две специфические особенности: диапазон NDC по оси Z равен `[0, 1]`, что решается макросом `GLM_FORCE_DEPTH_ZERO_TO_ONE`, и инвертированное направление оси Y в NDC компенсируется умножением элемента `proj[1][1]` на -1.

Процедурные цвета вершин вычисляются прямо в вершинном шейдере по формуле нормализации локальной позиции в диапазон `[0, 1]`: каждая компонента цвета получается из соответствующей координаты вершины с учетом границ пирамиды. Затем во фрагментном шейдере процедурный цвет умножается на цвет, который пользователь выбирает через `ColorEdit`, что позволяет одновременно видеть градиент по вершинам и управлять общим оттенком фигуры. Для отрисовки нескольких объектов создаются отдельные `uniform`-буферы и отдельные наборы дескрипторов (`VkDescriptorSet`) по одному на каждый объект, а в командном буфере для каждого объекта выполняется привязка своего набора дескрипторов и вызов `VkCmdDrawIndexed`. Такой подход позволяет использовать общую геометрию пирамиды, но задавать каждому объекту собственную модельную матрицу и цвет.

Используемые объекты Vulkan:

- 1) `VkBuffer`, `VmaAllocation` – вершинный, индексный, `uniform`-буферы;
- 2) `VkDescriptionSetLayout`, `VkDescriptorPool`, `VkDescriptorSet` – описание и привязка `uniform`-буферов к шейдерам;
- 3) `VkPipelineLayout`, `VkPipeline` – конфигурация графического конвейера: шейдеры, растеризация, `depth test`;
- 4) `VkRenderPass`, `VkFramebuffer`, `VkCommandBuffer` – `render pass` и запись команд отрисовки;
- 5) `VkShaderModule` – загрузка скомпилированных шейдеров;
- 6) `ImGui` – графический интерфейс для управления параметрами;
- 7) `GLFW` – создание окна и обработка ввода.

Результаты

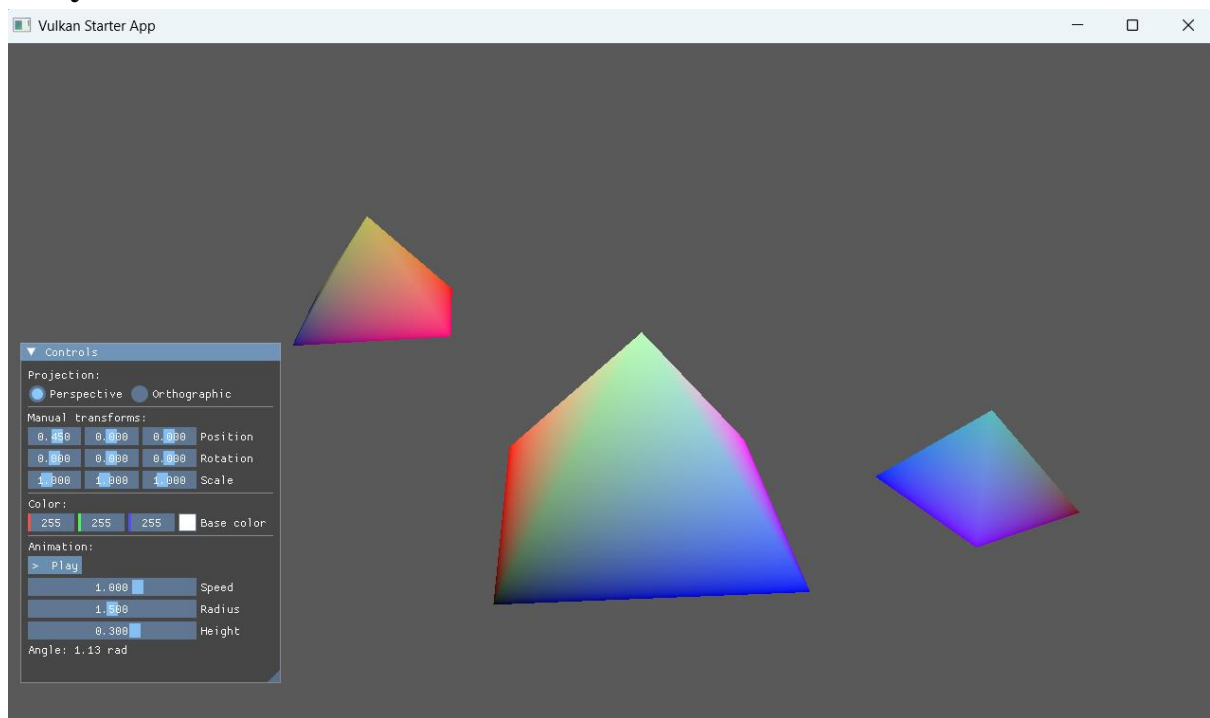


Рис. 1. Общий вид сцены: три пирамиды (управляемая и две статичные), панель управления в ImGui с настройками проекции, трансформацией, цвета и анимации. Перспективная проекция.

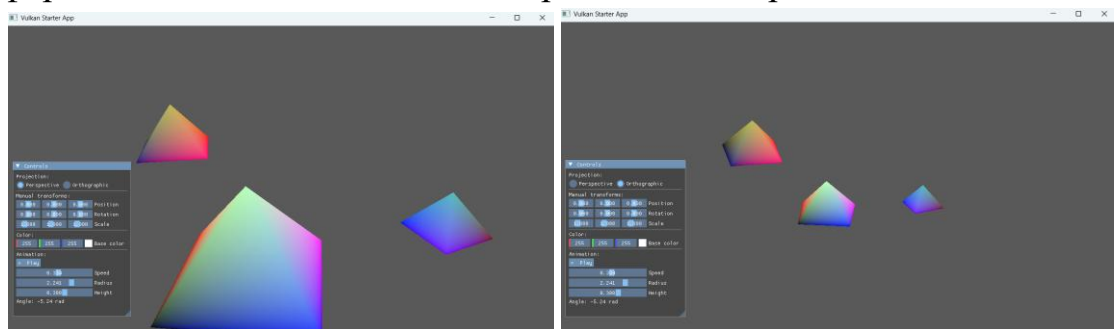


Рис. 2. Переключение проекции: слева – перспективная, справа – ортогографическая. Переключение выполняется радиокнопками в интерфейсе.

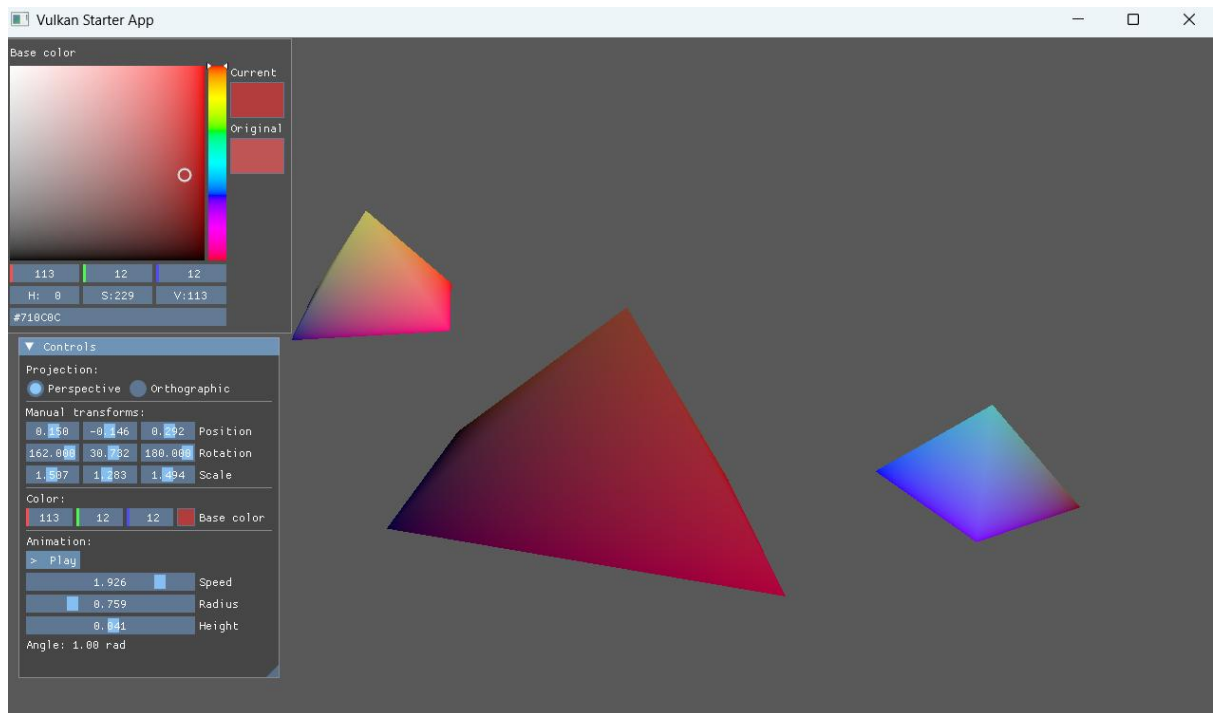


Рис. 3. Процедурные цвета вершин: каждая вершина окрашивается по формуле, цвет умножается на выбранный в ColorEdit. Виден градиент по каждой оси и изменение цвета при выборе красного множителя. Демонстрация переноса, поворота, изменения масштаба пирамиды.

Выводы

В ходе лабораторной работы я познакомилась с основами 3D-графики и получила практический опыт работы с ключевыми объектами API Vulkan. Научилась создавать и настраивать графический конвейер: загружать скомпилированные шейдеры, описывать входные атрибуты вершин, настраивать стадии растеризации, связывать пайплайн с render pass и дескрипторами. Разобралась с работой буферов через библиотеку VMA – созданием вершинного, индексного и uniform-буферов, копированием данных с CPU и обеспечением их доступности для GPU. Освоила математику 3D-преобразований: научилась строить модельную, видовую и проекционную матрицы, понимать разницу между перспективной и ортографической проекциями и применять их в шейдере в правильном порядке. Кроме того, научилась строить интерактивный интерфейс на ImGui: использовать слайдеры для управления трансформациями, радиокнопки для переключения проекции, ColorEdit для изменения цвета и кнопки для управления анимацией. Также были выполнены все задания лабораторной работы, включая шесть дополнительных: программа

работает в реальном времени, поддерживает динамическую смену проекции, трансформации, анимацию по траектории, процедурные цвета и отрисовку нескольких объектов на сцене.